CEBANC:

CURSO DE ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL Y BIG DATA

Modelos de inteligencia artificial · Unidad 5

ViviendaSeg

Sistema experto para la evaluación orientativa del riesgo de viviendas

Informe técnico y memoria del proyecto

AUTORES

Julen Altuna

Carmelo José Beato Garcia

Aplicación: viviendaseg-sistema-experto-587532865330.europe-west3.run.app

Código: github.com/raineragoge/ViviendaSeg-AIStudio

2 de noviembre de 2026

Índice

1. Resumen y caso de uso 3
    1.1 Resumen ejecutivo 3
    1.2 Problema, decisión que apoya y usuarios 3
    1.3 Alcance 3
    1.4 Equipo y herramientas 4
2. Organización y planificación 5
    2.1 Tareas, responsables y tiempos 5
3. Conocimiento y fuentes 6
    3.1 Representación del conocimiento 6
    3.2 Catálogo de reglas 6
    3.3 Fuentes y su relación con el conocimiento 7
    3.4 Vacíos y contradicciones detectados y corregidos 7
    3.5 Cómo añadimos información nueva 8
4. Razonamiento y funcionamiento 9
    4.1 Política de inferencia 9
    4.2 Ejemplo verificable 9
    4.3 Gestión de límites 9
5. Stack tecnológico y arquitectura 10
    5.1 Componentes del sistema experto 10
    5.2 Flujo de datos 10
    5.3 Justificación de las decisiones técnicas 11
6. Interfaz y explicaciones 12
    6.1 Capturas 12
7. Pruebas y validación 15
    7.1 Resumen de la validación automática 15
    7.2 Casos de prueba reproducibles 15
    7.3 Entradas no válidas 16
    7.4 Contraste con fuentes y evidencias 16
    7.5 Comparación con un LLM 16
8. Publicación, GitHub y reproducción 18
    8.1 Acceso y uso 18
    8.2 Reproducción local 18
    8.3 Contenido del README 18
9. Uso crítico de la IA 19
    9.1 Herramientas y aportación de cada una 19
    9.2 Registro de consultas relevantes 19
    9.3 Errores de la IA y correcciones 20
    9.4 Cómo se verificaron los resultados 20
10. Limitaciones y mejoras 21
    10.1 Limitaciones 21
    10.2 Mejoras propuestas 21
Anexo A. Conversación de diseño con ChatGPT 22
    A.1 Extractos de conversación 22
    A.2 Decisiones que salieron de la conversación 26
Anexo B. Código del motor de inferencia comentado 27
Referencias 28

1. Resumen y caso de uso

1.1 Resumen ejecutivo

ViviendaSeg es un sistema experto de IA simbólica que ayuda a un propietario a obtener una **evaluación orientativa del riesgo de su vivienda** (BAJO, MEDIO o ALTO) antes de plantearse la contratación de un seguro. A partir de un cuestionario adaptativo de hasta 19 preguntas, un motor de inferencia consulta un catálogo de **16 reglas SI–ENTONCES** (R01–R16) y devuelve el nivel de riesgo, las reglas que lo justifican, las recomendaciones asociadas y las respuestas pendientes de aclarar.

No es un modelo de Machine Learning ni utiliza puntuaciones acumuladas: la conclusión se obtiene por una **jerarquía de reglas** (un riesgo alto confirmado prevalece siempre) y la incertidumbre se muestra de forma explícita mediante el resultado **MEDIO PROVISIONAL**.

1.2 Problema, decisión que apoya y usuarios

Elemento	Definición
Problema	Los propietarios desconocen qué factores de su vivienda (instalaciones, estructura, conservación, ocupación) pueden suponer un riesgo y qué conviene revisar.
Decisión que apoya	Decidir qué revisiones o medidas preventivas priorizar antes de consultar un seguro. **No** calcula primas ni decide si una aseguradora aceptará la vivienda.
Información necesaria	Uso, año de construcción, estructura, instalación eléctrica, calefacción/gas, detectores, humedades, fugas, daños estructurales, mantenimiento, desocupación, seguridad e inundaciones.
Usuarios	Propietarios de vivienda habitual, segunda residencia o vivienda propia desocupada, sin conocimientos técnicos.
Criterio de éxito	Resultado coherente y reproducible con los casos de prueba; cada conclusión muestra la regla que la causa; los datos desconocidos no producen falsa certeza; la web es accesible públicamente.

1.3 Alcance

Incluido	Excluido
Vivienda propia habitual Segunda residencia de uso propio Vivienda propia desocupada Garajes y trasteros solo si están integrados en la vivienda	Viviendas destinadas al alquiler Oficinas, locales y establecimientos comerciales Valoraciones que exigen inspección profesional Cálculo de primas o probabilidades de siniestro Consulta catastral, mapas de inundación oficiales o verificación de documentos

El sistema es un **prototipo académico**: las reglas no han sido validadas por peritos, aseguradoras ni equipos de emergencias, y el resultado no debe usarse para decisiones reales de aseguramiento, seguridad o habitabilidad.

1.4 Equipo y herramientas

Este trabajo lo hemos hecho **Julen Altuna** y **Carmelo José Beato Garcia**. Hemos usado ChatGPT (para explorar el problema, diseñar y documentar), Google AI Studio (para desarrollar y publicar en Cloud Run), GitHub (código, README y comprobaciones automáticas) y Google Colab con SWI-Prolog (para validar la lógica).

2. Organización y planificación

Trabajamos de forma iterativa: planteábamos el problema, la IA hacía una propuesta, decidíamos nosotros, implementábamos, probábamos y corregíamos. Esta cronología la hemos reconstruido a partir de nuestro historial de trabajo y de la memoria técnica.

Fecha	Actividad	Resultado / cambio realizado
19/10	Definición del caso de uso, viviendas admitidas y excluidas, tres niveles de riesgo y tratamiento de la incertidumbre.	Decisión de no forzar respuestas definitivas sin datos esenciales.
20/10	Revisión de las reglas de construcción; preparación de motores y web.	Se corrige la idea de que toda estructura de madera es riesgo alto: depende de informes técnicos y circunstancias.
20–22/10	Desarrollo de la web, cuestionario condicional, explicaciones y separación base de conocimientos / motor.	Se distingue interfaz de lógica y se documenta cada activación.
22–23/10	Lógica Prolog y casos de prueba; 16 comprobaciones manuales en SWISH; validación en Colab.	La misma entrada debe activar la misma regla y poder explicarse.
28/10	Paridades, pruebas HTTP y adaptación a Google AI Studio; corrección de errores de despliegue.	No confundir que la vista previa cargue con que el despliegue funcione.
28/10	Cloud Run en estado Ready; caso ALTO comprobado desde ventana de incógnito; alineación de repositorios con PR y CI.	Publicación contrastada fuera del editor.
30/10	Dossier de traspaso del proceso de prompts (Carmelo).	Trazabilidad del proceso y lista de pendientes.

2.1 Tareas, responsables y tiempos

Esta tabla resume cómo organizamos las tareas y los plazos. Los tiempos son aproximados, ya que reconstruimos parte del seguimiento a partir de nuestro historial de trabajo.

Tarea (fase del trabajo)	Responsable	Plazo	Previsto	Real	Estado
Caso de uso y alcance (fases 1–2)	Julen y Carmelo	19/10	0,5h	0,5h	Hecho
Conocimiento y reglas (fases 5–7)	Julen y Carmelo	19–22/10	2h	1,5h	Hecho
Stack y prototipo (fases 8–10)	Julen y Carmelo	20–22/10	2h	2h	Hecho
Pruebas con casos difíciles (fase 11)	Julen y Carmelo	22–28/10	3h	2h	Hecho
Publicación y GitHub (fase 14)	Julen y Carmelo	28/10	0,3h	s/d	Hecho
Comparación con un LLM (fase 13)	Julen y Carmelo	02/11	0,3h	0,5h	Hecho
Memoria e informe final	Julen y Carmelo	30/10–02/11	1h	2h	Hecho

3. Conocimiento y fuentes

3.1 Representación del conocimiento

Separamos el conocimiento de la lógica del programa. El archivo knowledge.json reúne las preguntas, las reglas y las explicaciones que utiliza el sistema:

- **Hechos:** respuestas del usuario a 19 preguntas potenciales, organizadas en cinco bloques (Construcción, Instalaciones, Protección, Conservación, Ocupación y entorno). La mayoría de las preguntas permite responder «No lo sé»; el uso de la vivienda debe indicarse para determinar si entra en el alcance.
- **Reglas:** 16 reglas SI–ENTONCES con identificador, nivel (alto/medio), condiciones, motivo, recomendación y categoría. 5 reglas son de nivel alto y 11 de nivel medio.
- **Conclusiones:** nivel global (BAJO, MEDIO, MEDIO PROVISIONAL, ALTO o FUERA DE ALCANCE), reglas activadas con su explicación y datos pendientes.

3.2 Catálogo de reglas

ID	Nivel	Condición (SI)	Motivo mostrado al usuario
R01	ALTO	Estructura de madera + informe técnico desfavorable sobre resistencia al fuego	Estructura principal de madera con informe desfavorable.
R02	ALTO	Estructura mixta con madera principal + informe desfavorable	Estructura mixta con madera principal e informe desfavorable.
R03	ALTO	Instalación eléctrica con defectos conocidos	Defectos conocidos en la instalación eléctrica.
R04	ALTO	Calefacción de gas + fuga declarada o sospechada (urgente)	Posible fuga de gas.
R05	ALTO	Daños estructurales graves (urgente)	Daños que podrían afectar a la estabilidad.
R06	MEDIO	Instalación eléctrica sin revisión reciente	Sin revisión eléctrica reciente declarada.
R07	MEDIO	Calefacción de gas + mantenimiento pendiente	Mantenimiento del sistema de gas pendiente.
R08	MEDIO	Calefacción de combustión + mantenimiento de chimenea/estufa pendiente	Chimenea o estufa sin mantenimiento al día.
R09	MEDIO	Calefacción de combustión + sin detectores de humo	Combustión activa sin detectores de humo.
R10	MEDIO	Humedades o filtraciones graves	Filtraciones o humedades importantes.
R11	MEDIO	Fuga de agua activa	Fuga de agua activa.
R12	MEDIO	Conservación con reparaciones importantes pendientes	Reparaciones importantes pendientes.
R13	MEDIO	Desocupación prolongada	Largos periodos sin supervisión.
R14	MEDIO	Desocupación prolongada + sin alarma ni puerta reforzada	Desocupación combinada con falta de medidas contra intrusiones.
R15	MEDIO	Antecedentes de inundaciones	Antecedentes de inundación en el inmueble.
R16	MEDIO	Madera sin protección declarada + sin informe favorable (o desconocido)	Señal preventiva, no prueba de incumplimiento.

Tabla 1. Catálogo final de reglas R01–R16 (fuente: knowledge.json, versión 1.1.0).

3.3 Fuentes y su relación con el conocimiento

Recogemos las fuentes consultadas y las referencias localizadas durante el diseño. Las cuatro últimas se identificaron con ayuda de la IA, pero no se contrastaron directamente en el desarrollo:

Fuente	Enlace	Reglas relacionadas
Real Decreto 314/2006, Código Técnico de la Edificación (BOE)	boe.es/eli/es/rd/2006/03/17/314/con	R01, R02, R05, R16 (marco general de seguridad estructural y contra incendios)
CTE, Documento Básico DB-SI: Seguridad en caso de incendio	codigotecnico.org/DocumentosCTE/SeguridadEnCasoDeIncendio.html	R01, R02, R09, R16 (resistencia al fuego, detección)
CTE, DB-SI, anejo SI E: resistencia al fuego de estructuras de madera	codigotecnico.org (documento SI E)	R01, R02, R16 (la madera puede cumplir sin revestimientos adicionales)
Recomendaciones del Ayuntamiento de Donostia en emergencias	donostia.eus (seguridad ciudadana y emergencias)	R04, R05 (qué hacer ante fuga de gas o daño estructural)
Guía de prevención de incendios en el hogar (Fundación MAPFRE y bomberos)	Localizada en la conversación de diseño (Anexo A)	R03, R06, R09 (riesgo eléctrico y detectores)
Ministerio del Interior: prevención de robos	Anexo A	R13, R14 (ausencias y seguridad frente a intrusiones)
MITECO: cartografía de zonas inundables	Anexo A	R15 (la ausencia de cartografía no demuestra ausencia de riesgo)
UNESPA: estadísticas de siniestros	Anexo A	Contexto general: relevancia de los daños por agua e incendios

Tabla 2. Fuentes del proyecto. Las cuatro últimas las localizamos en la conversación de diseño y no están enlazadas en el README.

Vacío detectado: las reglas R07 y R08 (gas y combustión) y R10–R12 (humedades, fugas y mantenimiento) no tienen una norma específica enlazada; son criterios preventivos orientativos que todavía requieren un contraste documental más específico. Para reforzarlas proponemos consultar el RITE (RD 1027/2007) y el reglamento de instalaciones de gas para R04 y R07, y el Reglamento Electrotécnico de Baja Tensión (RD 842/2002) para R03 y R06. No las hemos consultado durante el desarrollo, así que las citamos como mejora y no como evidencia.

3.4 Vacíos y contradicciones detectados y corregidos

- **Madera = riesgo alto (descartada).** Al principio marcábamos toda estructura de madera como ALTO. Lo corregimos: la madera no determina el nivel; preguntamos por protecciones y por documentación técnica (favorable, desfavorable, sin informe o desconocido). R01/R02 exigen informe desfavorable y R16 solo emite una señal preventiva.
- **Gas «sin revisión» como alto (descartada).** En una primera propuesta de pruebas, «gas sin revisión conocida» daba ALTO. Mantuvimos el criterio implementado: mantenimiento pendiente es MEDIO (R07), «no lo sé» es MEDIO PROVISIONAL y solo la fuga declarada activa ALTO (R04). Queda como punto a revisar con el docente.
- **Respuestas de ramas incompatibles.** Una respuesta sobre caldera de gas con calefacción eléctrica no debe activar R04. El motor descarta los hechos de preguntas no visibles (caso C10 de las pruebas).
- **Año de construcción.** No se usa como criterio automático: solo activa la pregunta sobre reformas (viviendas anteriores a 2006).

3.5 Cómo añadimos información nueva

La información nueva entra en el sistema añadiendo **reglas de cribado**: cada una es una condición SI–ENTONCES que, si se cumple, activa un nivel de riesgo (alto o medio) con su motivo y su recomendación. En Prolog, que es como la tenemos escrita de forma más directa y legible, una regla es una línea del fichero rules.pl:

```
% Ejemplo (no forma parte del catálogo): gas sin detectores de humo -> riesgo medio
regla(r17, medio) :- dato(calefaccion, gas), dato(detectores, no).
```

Los pasos para incorporar una regla son:

1. Definir la condición con los hechos que ya pregunta el cuestionario (o añadir la pregunta nueva si hace falta) y su nivel.
2. Escribir la regla en rules.pl (versión Prolog) y, para que la aplique la web, añadir la misma regla en knowledge.json con su ID, condiciones, motivo y recomendación.
3. Ejecutar python scripts/sync_knowledge.py para regenerar la base de conocimientos que lee el servidor, y las pruebas de paridad (test_parity.py y test_prolog_parity.py) para comprobar que los motores siguen coincidiendo.
4. Publicar de nuevo; la regla aparece automáticamente en el informe con su ID, su motivo y su recomendación.

Hay que tener presente que la aplicación publicada **ejecuta el motor JavaScript** con knowledge.json; Prolog lo usamos para escribir y validar las reglas, pero la web no lo invoca en cada evaluación (apartado 5).

4. Razonamiento y funcionamiento

4.1 Política de inferencia

El motor aplica una **lógica jerárquica, no una suma de puntos**, para que un factor peligroso no quede compensado por varios factores positivos:

1. Validar el uso de la vivienda; si es alquiler u otro uso no residencial, devolver **FUERA DE ALCANCE**. Si falta el uso, devolver «incompleto».
2. Normalizar las respuestas, validar valores permitidos y descartar los hechos de ramas que no corresponden.
3. Evaluar el catálogo fijo de reglas SI–ENTONCES y registrar una traza (regla → activada o no).
4. Si hay al menos una regla ALTA activada, devolver **ALTO** (aunque falten otros datos).
5. Si no hay alto y falta algún dato esencial, devolver **MEDIO PROVISIONAL**.
6. Si hay alguna regla MEDIA y no faltan datos esenciales, devolver **MEDIO**.
7. Si no se activa ninguna regla y no faltan datos esenciales, devolver **BAJO**.
8. Incorporar IDs de reglas, motivos, recomendaciones y preguntas pendientes en la respuesta.

4.2 Ejemplo verificable

Entrada: uso = habitual, estructura = hormigón, electricidad = defectuosa, calefacción = eléctrica y el resto de respuestas esenciales favorables. Resultado: **ALTO**, provisional = falso, regla **R03**. Los demás factores favorables no reducen la condición alta.

4.3 Gestión de límites

Situación	Comportamiento del sistema
Datos incompletos	MEDIO PROVISIONAL con la lista de preguntas pendientes (salvo que ya haya un ALTO confirmado).
Respuestas incompatibles / ramas ocultas	Se descartan los hechos de preguntas no visibles; no activan reglas.
Contradicción entre factores	Prevalece el riesgo alto confirmado sobre cualquier factor favorable o desconocido.
Caso no previsto (alquiler, local)	Resultado FUERA DE ALCANCE con explicación; no se evalúa.
Entradas mal formadas	La API responde con error (campo desconocido, opción no válida, año fuera de rango, cuerpo > 30 000 bytes).
Madera con documentación no contrastable	Se advierte que el sistema no verifica documentos; sin informe → provisional.

5. Stack tecnológico y arquitectura

5.1 Componentes del sistema experto

Componente	Implementación	Fichero
Interfaz de usuario	Cuestionario adaptativo por bloques, ejemplos precargados e informe imprimible (HTML/CSS/JavaScript).	public/index.html, style.css, main.js
Base de conocimientos	Preguntas, reglas, motivos, recomendaciones y referencias en JSON; copia generada para el servidor.	knowledge.json → functions/_shared/knowledge.js
Motor de inferencia	Evaluación jerárquica de reglas SI–ENTONCES y gestión de datos desconocidos.	functions/_shared/engine.js
Módulo de explicaciones	Reglas activadas con motivo, recomendación, traza y preguntas pendientes.	Respuesta de /api/evaluate + main.js
Adquisición y mantenimiento del conocimiento	Actualizamos reglas y preguntas en el JSON y regeneramos la copia que utiliza el motor; después comprobamos su coherencia mediante pruebas.	knowledge.json; scripts/sync_knowledge.py

Tabla 3. Los cinco componentes del sistema experto. La ejecución en Node.js y la ausencia de almacenamiento se explican en los apartados 5.2 y 5.3.

5.2 Flujo de datos

```
Navegador (public/index.html + main.js)
        │  GET /api/schema            POST /api/evaluate (JSON)
        ▼
Servidor HTTP Node.js (server.js)  ── escucha en 0.0.0.0:PORT (8080 por defecto)
        │
        ▼
Motor de inferencia (functions/_shared/engine.js)
        │  lee
        ▼
Base de conocimientos (functions/_shared/knowledge.js ← knowledge.json)
        │
        ▼
Respuesta: level · provisional · triggered_rules · missing · recommendations · trace
```

5.3 Justificación de las decisiones técnicas

Decisión	Justificación
Node.js nativo sin dependencias	Cloud Run/AI Studio solo necesitan un servidor que escuche en PORT; evita instalar Python o Prolog en producción y simplifica la reproducción (npm start).
Motor de reglas en JavaScript	Reutiliza una única implementación en servidor y pruebas; se verificó su equivalencia con Python en 507 escenarios.
Conocimiento separado en JSON	Permite editar reglas sin tocar la interfaz; sync_knowledge.py --check garantiza que la copia JS está sincronizada.
Python y Prolog como validación	Dos implementaciones independientes sirven para contrastar la lógica (paridad). **La web no ejecuta Prolog ni Python en producción.**
Sin base de datos	No hay necesidad de almacenar respuestas; reduce riesgos de privacidad (no se pide dirección ni referencia catastral).
Cabeceras de seguridad	CSP restrictiva (script-src 'self'), X-Content-Type-Options: nosniff y Cache-Control: no-store; límite de 30 000 bytes por petición.
Sin claves en el repositorio	El servidor no necesita secretos; no se incluyen claves ni contraseñas.

Coherencia diagrama–código: el flujo anterior coincide con server.js (importa schema y evaluate desde functions/_shared/engine.js). Los ficheros functions/api/*, GUIA_PUBLICACION.md y el flujo de trabajo de Cloudflare pertenecen al despliegue histórico en Cloudflare Pages y no se usan en Cloud Run.

6. Interfaz y explicaciones

La aplicación pública (comprobada el 02/11/2026) muestra una cabecera con la indicación «Motor de inferencia · JavaScript», un aviso de uso académico y un asistente por pasos con cinco bloques temáticos y barra de progreso (13 preguntas visibles en el recorrido base).

- **Preguntas claras con ayuda contextual: la mayoría incluye la opción «No lo sé», que se trata como incertidumbre y no como error. El uso de la vivienda debe indicarse para comprobar el alcance.**
- **Preguntas condicionales:** solo aparecen cuando procede (p. ej., la pregunta de caldera solo si la calefacción es de gas; las de madera solo con estructura de madera o mixta con madera principal).
- **Casos precargados:** botones «Caso de riesgo bajo / medio / alto» y «Datos desconocidos» para comprobar las reglas; las respuestas pueden revisarse y modificarse.
- **Explicación del resultado:** nivel global, indicador de provisionalidad, reglas activadas (ID, motivo y recomendación), avisos urgentes (R04, R05), respuestas pendientes e informe imprimible.
- **Transparencia:** panel con 16 reglas, prioridad jerárquica, incertidumbre explícita y explicaciones trazables; indicación de que no se guardan respuestas.

6.1 Capturas

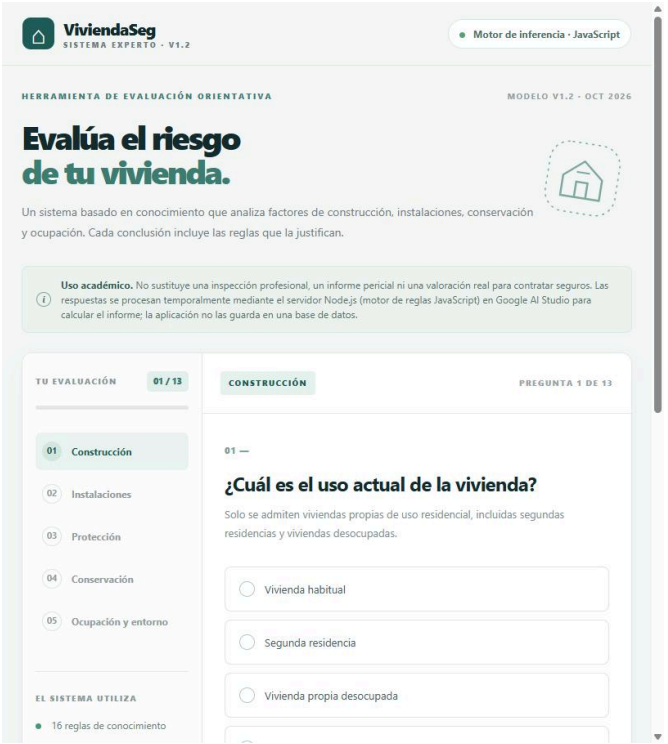


Figura 1. Pantalla de inicio de la aplicación publicada en Google AI Studio / Cloud Run (captura del 02/11/2026).

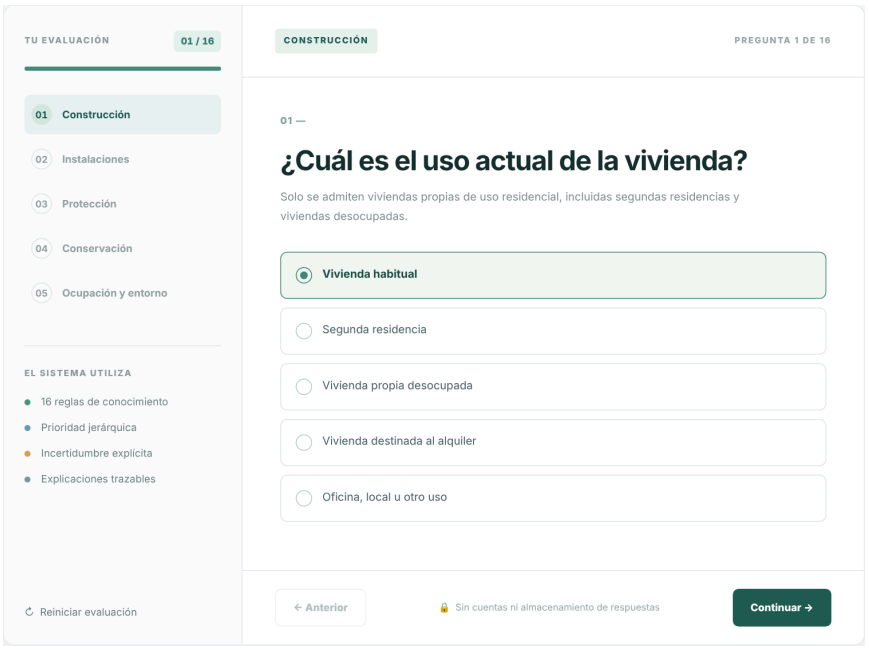


Figura 2. Cuestionario adaptativo con el caso de ejemplo «riesgo alto» cargado: 16 preguntas por las ramas condicionales de madera.

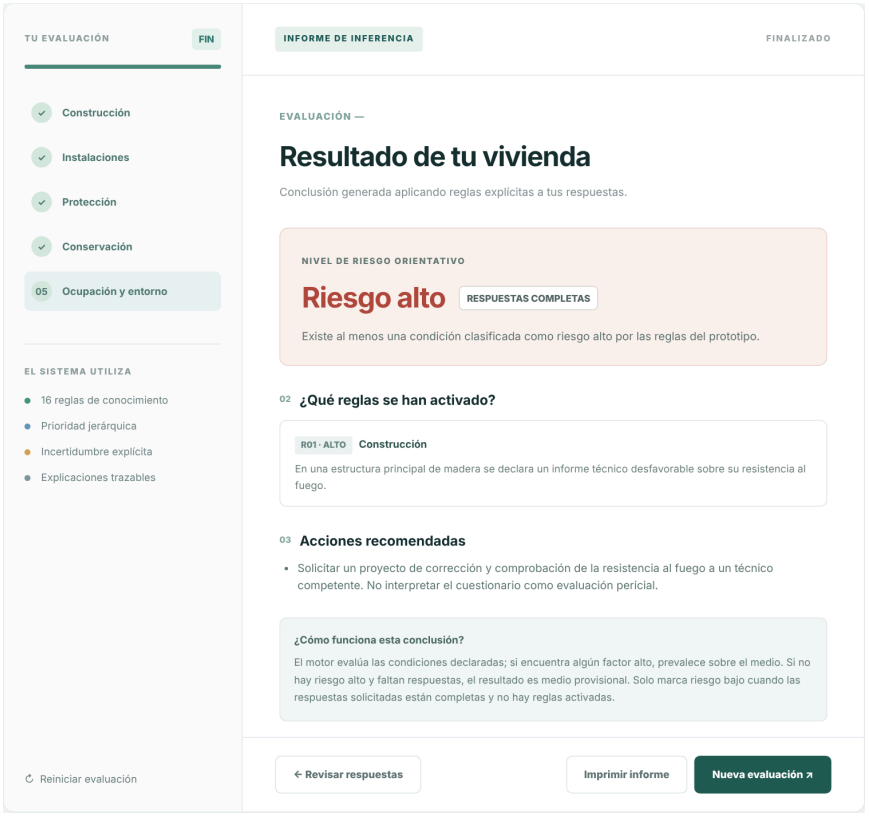


Figura 3. Resultado ALTO con la regla R01 que lo justifica, su motivo y su recomendación.



Figura 4. Resultado MEDIO PROVISIONAL con el listado de información pendiente.

Las capturas de las figuras 2 a 4 las hemos sacado ejecutando en local el mismo código del repositorio ViviendaSeg-AIStudio (versión 1.2, npm start), con el mismo motor y la misma base de conocimientos que la aplicación publicada.

7. Pruebas y validación

7.1 Resumen de la validación automática

Conjunto	Resultado	Interpretación
Pruebas JavaScript (integración y API)	14 superadas	El motor y la API responden a los casos y contratos probados. Reejecutadas en la preparación de este informe: 14/14.
Pruebas unitarias Python	40 superadas	El motor de referencia produce las salidas esperadas. Reejecutadas: 40/40, OK.
Paridad Python–JavaScript	507 escenarios coincidentes	Sin discrepancias entre ambas implementaciones. Reejecutada: OK. sync_knowledge.py --check: sincronizado.
Prolog directo (SWISH y Colab)	16 pruebas superadas	Reglas y exclusiones concretas comprobadas en el evaluador declarativo.
Paridad Python–Prolog (Colab, 23/10)	514 escenarios coincidentes	14 dirigidos + 500 aleatorios reproducibles; se compara nivel, provisionalidad, reglas y pendientes.
HTTP / CI / producción	API y servidor probados; Cloud Run «Ready»; caso ALTO desde incógnito	El servicio respondió fuera del editor para el caso comprobado.

Tabla 4. Resultados registrados. La coincidencia entre motores mide coherencia de implementación, no validez actuarial.

7.2 Casos de prueba reproducibles

En la tabla están los casos habituales y difíciles que probamos. Base común: uso habitual, 2018, hormigón, electricidad revisada, calefacción eléctrica, detectores sí, sin humedades, sin fugas, sin daños, buen mantenimiento, sin desocupación, alarma y puerta reforzada, sin inundaciones. Los resultados «obtenidos» los calculamos ejecutando el motor del repositorio (evaluate) el 02/11/2026.

Nº	Caso (cambio sobre la base)	Esperado	Obtenido	Regla / pendiente	OK
C1	Base completa, todo favorable	BAJO	BAJO (no prov.)	—	✔
C2	Electricidad sin revisión reciente	MEDIO	MEDIO	R06	✔
C3	Electricidad con defectos	ALTO	ALTO	R03	✔
C4	Electricidad «No lo sé»	MEDIO PROVISIONAL	MEDIO prov.	Falta: electricidad	✔
C5	Vivienda destinada al alquiler	FUERA DE ALCANCE	excluded	—	✔
C6	Gas + fuga declarada	ALTO (urgente)	ALTO	R04	✔
C7	Madera + protección + informe favorable	BAJO	BAJO	—	✔
C8	Madera + informe desfavorable	ALTO	ALTO	R01	✔
C9	Madera sin protección y sin informe	MEDIO PROVISIONAL	MEDIO prov.	R16; falta informe_madera	✔
C10	Calefacción eléctrica pero «caldera = fuga» (rama oculta)	BAJO (dato descartado)	BAJO	R04 no se activa	✔
C11	Eléctrica defectuosa + detectores y humedades desconocidos	ALTO prevalece	ALTO (no prov.)	R03; faltan detectores, humedades	✔
C12	Desocupada, sin alarma ni puerta	MEDIO	MEDIO	R13, R14	✔
C13	Formulario vacío (sin uso)	Incompleto	incomplete / medio prov.	Falta: uso	✔

Nº	Caso (cambio sobre la base)	Esperado	Obtenido	Regla / pendiente	OK
C14	Solo se indica el uso	MEDIO PROVISIONAL	MEDIO prov.	12 datos pendientes	✔
C15	Estructura mixta sin madera principal	BAJO (sin preguntas de madera)	BAJO	—	✔
C16	Mixta con madera + informe desfavorable	ALTO	ALTO	R02	✔
C17	Calefacción de combustión, chimenea pendiente, sin detectores	MEDIO	MEDIO	R08, R09	✔

Tabla 5. Casos de prueba: entrada, resultado esperado y resultado obtenido (motor JavaScript del repositorio).

7.3 Entradas no válidas

Entrada	Respuesta de la API
{"foo": 1}	Error 400: «Campo desconocido: foo»
{"electricidad": "xx"}	Error 400: «Respuesta no válida en electricidad»
{"anio": 1500}	Error 400: «El año debe ser un entero válido o 'unknown'»

7.4 Contraste con fuentes y evidencias

- Las reglas de madera se contrastaron con el DB-SI del CTE (la madera puede cumplir la resistencia al fuego; no se presume incumplimiento).
- Registro de las 16 comprobaciones manuales en SWISH: docs/VALIDACION_PROLOG_2026-10-23.md (con la incidencia del escenario 12, que se repitió al devolver la salida del escenario previo).
- Cuaderno de Colab notebooks/validacion_python_prolog.ipynb: salida con código 0 para las 16 pruebas y los 514 escenarios.
- Integración continua (GitHub Actions, validar-aistudio.yml): comprueba sintaxis, pruebas JS y Python, paridad, sincronización del conocimiento y servidor HTTP en el puerto 8080 con los casos bajo, medio, alto y provisional.
- Prueba externa: Cloud Run «Ready» y caso ALTO completado en ventana de incógnito (28/10/2026).

7.5 Comparación con un LLM

Probamos la misma tarea en un modelo conversacional para ver qué aporta nuestro sistema experto. **Modelo:** Claude (Sonnet 5.5), en una instancia nueva sin acceso al repositorio, a las reglas ni a los resultados del sistema. **Prompt:** «Evalúa el riesgo de esta vivienda como BAJO, MEDIO o ALTO y justifica brevemente. Indica si falta algún dato para estar seguro», con los mismos datos de cada caso expresados en lenguaje natural. **Casos:** C1, C3, C4, C6, C9, C10 y C11 de la Tabla 5, una ejecución por caso (02/11/2026).

Caso	Sistema experto	LLM (Claude)	Coincide	Observaciones
C1 Todo favorable	BAJO	BAJO	Sí	Añade datos fuera del alcance (ubicación, sismo, valor asegurado).
C3 Eléctrica defectuosa	ALTO · R03	ALTO	Sí	Justifica bien, pero sin referirse a una regla identificable.

Caso	Sistema experto	LLM (Claude)	Coincide	Observaciones
C4 Dato desconocido	MEDIO PROVISIONAL	MEDIO	Sí	Trata la incertidumbre en texto libre; no la marca como estado propio.
C6 Fuga de gas	ALTO · R04 (urgente)	ALTO	Sí	Indica actuación urgente y señala que el detector de humo no detecta gas.
C9 Madera sin informe	MEDIO PROVISIONAL · R16	ALTO	No	El LLM presume vulnerabilidad grave; el sistema no presume incumplimiento (DB-SI).
C10 Rama incompatible	BAJO (descarta el dato)	ALTO	No	El LLM asume el peor caso ante la contradicción y pide aclararla.
C11 Alto + desconocidos	ALTO · R03	ALTO	Sí	Coincide; el LLM mezcla riesgos (humedad + cortocircuito) sin regla explícita.

Tabla 6. Comparación de los niveles básicos de riesgo (BAJO, MEDIO y ALTO). En C4 existe coincidencia en MEDIO, pero no en la provisionalidad.

Resultado: coinciden en la categoría básica de riesgo en 5 de los 7 casos, pero no necesariamente en el resultado completo. En C4, Claude responde MEDIO sin marcar un estado provisional estructurado. En C9, Claude clasifica el caso como ALTO y nuestro sistema como MEDIO PROVISIONAL; sin una comprobación técnica no podemos afirmar cuál de los dos refleja mejor el riesgo real. En C10, Claude pide aclarar un dato contradictorio, mientras que nuestro motor lo descarta por pertenecer a una pregunta que no debería estar visible. Es una diferencia de tratamiento de los datos, no una comparación directa de la misma entrada efectiva.

Qué aporta el sistema desarrollado: (1) determinismo: la misma entrada produce siempre la misma salida, algo que el LLM no garantiza; (2) trazabilidad: cada conclusión remite a una regla con ID, motivo y recomendación, mientras que el LLM justifica en texto libre sin fuentes comprobables; (3) incertidumbre como estado propio (MEDIO PROVISIONAL) con la lista exacta de datos pendientes; (4) alcance acotado: no añade preguntas ni factores ajenos; (5) conocimiento editable y versionado, con pruebas automáticas. **Qué aporta el LLM:** flexibilidad para entradas en lenguaje natural, detección de contradicciones y explicaciones más ricas, a costa de variabilidad y falta de trazabilidad.

Limitaciones del experimento: un solo modelo (Claude, no ChatGPT ni Gemini), una única ejecución por caso (no se mide la consistencia entre repeticiones) y un enunciado en lenguaje natural redactado por nosotros. Debe leerse como contraste cualitativo, no como una evaluación estadística.

8. Publicación, GitHub y reproducción

Entregable	Enlace
Aplicación pública (Google AI Studio / Cloud Run)	https://viviendaseg-sistema-experto-587532865330.europe-west3.run.app/
Repositorio de la versión final (AI Studio)	https://github.com/raineragoge/ViviendaSeg-AIStudio
Repositorio original y material Prolog	https://github.com/raineragoge/ViviendaSeg-Sistema-Experto
PR de alineación	https://github.com/raineragoge/ViviendaSeg-AIStudio/pull/1
Ejecución de CI de referencia	https://github.com/raineragoge/ViviendaSeg-AIStudio/actions/runs/40676981948
Registro de validación Prolog	https://github.com/raineragoge/ViviendaSeg-AIStudio/blob/main/docs/VALIDACION_PROLOG_2026-10-23.md
Proyecto en Google AI Studio	https://aistudio.google.com/apps/ad944814-b2c4-44b2-ad93-8751cb078a22

Repositorios: el código de la **versión publicada** (servidor server.js para Cloud Run) está en ViviendaSeg-AIStudio. ViviendaSeg-Sistema-Experto es el repositorio anterior, con el despliegue en Cloudflare Pages y el material Prolog.

8.1 Acceso y uso

Basta con abrir el enlace público, responder al cuestionario (o cargar un caso de ejemplo) y consultar el informe con el nivel, las reglas activadas, las recomendaciones y los datos pendientes. No hace falta cuenta. Comprobamos el acceso externo el 28/10/2026 desde una ventana de incógnito (caso ALTO) y el 02/11/2026 volvimos a verificar que la página pública carga con el cuestionario y los cuatro casos de ejemplo.

8.2 Reproducción local

```
# Requisito: Node.js 20 o superior (sin dependencias de terceros)
git clone https://github.com/raineragoge/ViviendaSeg-AIStudio
cd ViviendaSeg-AIStudio
npm start            # servidor en http://localhost:8080
npm test             # 14 pruebas JavaScript

# Validación académica opcional (Python 3.9+)
npm run test:python
npm run test:parity
python scripts/sync_knowledge.py --check
```

8.3 Contenido del README

El README del repositorio de AI Studio recoge objetivo, destinatarios, acceso, arquitectura, componentes, casos de prueba con resultado esperado, pruebas automatizadas, organización de ficheros, protección y limitaciones, y enlaces. Nos queda añadir una sección de «ejemplo de uso» con capturas.

9. Uso crítico de la IA

9.1 Herramientas y aportación de cada una

Herramienta	Uso y aportación
ChatGPT	Exploración del problema, definición de alcance, propuesta de reglas y política de inferencia, redacción de la memoria y diagnóstico de errores de despliegue.
Google AI Studio	Desarrollo técnico (importación y adaptación del servidor) y publicación en Cloud Run.
GitHub / Actions	Control de versiones, PR y comprobaciones automáticas.
Colab + SWI-Prolog	Ejecución real de Prolog para validar la lógica y su paridad con Python.
Claude	Nos ayudó a montar este informe a partir del repositorio y de nuestros apuntes. Recalculamos los datos de pruebas ejecutando el código y con él hicimos la comparación con un LLM (apartado 7.5).

9.2 Registro de consultas relevantes

Estos prompts son formulaciones reconstruidas a partir de nuestro historial y de la memoria técnica; no son transcripciones literales. En el Anexo A incluimos una selección de conversaciones que muestra las decisiones más relevantes sin cargar la memoria con todo el historial.

ID	Fecha	Petición a la IA (resumen)	Decisión propia / corrección	Verificación
P01	19/10	Definir problema, destinatarios, casos incluidos y excluidos, categorías de riesgo y salida; sin precios de pólizas.	Alcance de viviendas propias; finalidad educativa.	Alcance documentado en README.
P02	19–22/10	Organizar como sistema experto con interfaz, base de conocimientos, motor, explicaciones y adquisición del conocimiento; no ML.	Se fijan los cinco componentes.	Localización en código (Tabla 3).
P03	19–20/10	Definir preguntas y reglas SI–ENTONCES con ID, nivel, motivo y recomendación.	Catálogo R01–R16 y 19 preguntas.	Revisión de reglas y pruebas.
P04	19–20/10	Criterio jerárquico sin puntuación acumulada; provisional si faltan datos.	Rechazo de la suma de puntos.	Casos alto, medio, bajo y provisional.
P05	20/10	Revisar la regla «madera = alto»; introducir informe técnico y protecciones.	Se corrige un error de diseño de la IA/propio.	R01, R02, R16 y casos C7–C9, C16.
P06	20–22/10	Cuestionario condicional y explicaciones finales.	Evitar preguntas irrelevantes.	Prueba manual de ramas.
P07	20–22/10	Separar datos, reglas y motores; API de esquema y evaluación; Python y Prolog.	Arquitectura real descrita sin exagerar.	Tests de API.
P08	22–23/10	Escenarios verificables y comprobación Prolog/Python.	Detectar fallos de prioridad.	16 comprobaciones manuales.
P09	23–28/10	Adaptador Python → Prolog → Python y comparación.	Exigir ejecución real, no solo archivo.	514 escenarios en Colab.
P10	20–22/10	Preparar despliegue web (Cloudflare Pages); no afirmar Prolog en producción.	Primer despliegue.	API verificada.
P11	28/10	Diagnosticar errores de AI Studio/Cloud Run (wrangler, python3, PORT).	Pasar a servidor Node.js nativo sin tocar la lógica.	Cloud Run Ready.
P12	28/10	No fiarse del estado «Ready»: probar incógnito con caso ALTO.	Evidencia externa.	Prueba personal en incógnito.

ID	Fecha	Petición a la IA (resumen)	Decisión propia / corrección	Verificación
P13	28/10	Alinear GitHub con la versión desplegada, CI y PR.	Preservar el repositorio original.	PR #1 y GitHub Actions.
P14	28/10	Redactar memoria con pruebas, límites y uso de IA; no inventar feedback, horas ni comparación con LLM.	Dejar pendientes identificados.	Revisión del contenido frente al código.

9.3 Errores de la IA y correcciones

Problema observado	Corrección	Lección
Madera clasificada automáticamente como ALTO	Informes de resistencia al fuego y estado desconocido.	No convertir supuestos en hechos técnicos.
Preguntas de una rama incompatible	Descartar respuestas irrelevantes según calefacción y uso.	La coherencia de hechos importa tanto como las reglas.
Prolog como archivo sin ejecución demostrada	Pruebas directas y adaptador JSON Python–Prolog en Colab.	Código existente ≠ comprobación real.
Confusión entre vista previa y publicación	Revisar registros, estado de Cloud Run y acceso en incógnito.	No afirmar «listo» solo porque el editor lo diga.
«python3: not found» y fallo de puerto	Servidor Node.js que escucha en 0.0.0.0:PORT.	Adaptar el despliegue al runtime real.
Archivos de GitHub aún describían Cloudflare	PR con scripts, README, pruebas y CI.	Documentación y código deben reflejar la versión desplegada.
Escenario 12 de SWISH devolvió la salida del anterior	Se repitió con consulta completa y se contabilizó una sola vez.	Verificar cada salida copiada.

9.4 Cómo se verificaron los resultados

No dimos por correctas las propuestas de la IA solo porque parecieran razonables. Comprobamos el funcionamiento de las reglas con JavaScript, Python y Prolog, contrastamos con el CTE los criterios relativos a la estructura de madera y probamos la web fuera del editor. También revisamos que el README describiera lo que realmente publicamos. Aun así, algunas reglas preventivas requieren más fuentes específicas y no están validadas por especialistas.

Probamos también casos límite: viviendas con casi todas las respuestas favorables, pero con un problema importante. Por ejemplo, una instalación eléctrica con defectos conocidos o una fuga de gas declarada deben activar ALTO aunque el resto esté bien. En cambio, una estructura de madera sin deficiencias acreditadas no se clasifica automáticamente como riesgo alto.

10. Limitaciones y mejoras

10.1 Limitaciones

- Reglas simplificadas sin validación de peritos, aseguradoras o servicios de emergencia; uso exclusivamente educativo.
- El sistema **no verifica** documentos ni inspecciona la vivienda; una afirmación del usuario no equivale a una certificación.
- No calcula probabilidades ni primas; la coincidencia entre motores no mide validez actuarial.
- Varias reglas carecen de una norma específica enlazada (apartado 3.3).
- La web no ejecuta Prolog; la integración Python–Prolog solo se usó para validar.
- No se ha realizado auditoría externa de seguridad ni de la configuración de registros del proveedor.

10.2 Mejoras propuestas

- Vincular cada regla a una fuente normativa concreta (REBT, RITE, reglamento de gas).
- Integrar la consulta catastral y la cartografía oficial de inundabilidad.
- Repetir la comparación con otros modelos (ChatGPT, Gemini) y con varias repeticiones por caso.
- Pruebas de usabilidad con personas externas y ampliación de casos de prueba.

Anexo A. Conversación de diseño con ChatGPT

A.1 Extractos de conversación

USUARIO

se termino la clase, ahora haremos este trabajo: [archivo cargado con la tarea y la rúbrica]

CHATGPT

Me parece una mejora muy interesante, especialmente porque permitiría que nuestro sistema experto recopilase automáticamente parte de los hechos necesarios para razonar, en lugar de depender únicamente de las respuestas del usuario.

He comprobado que el Catastro español ofrece servicios de consulta de información no protegida. Podemos aprovecharlos, aunque hay algunas limitaciones que debemos tener presentes.

1. ¿Qué podríamos obtener con la referencia catastral? Identificación automática de la vivienda: el usuario introduce su referencia catastral y la aplicación intenta recuperar los datos públicos disponibles.

Dato	Disponibilidad
Ubicación y dirección	Consultable
Superficie	Consultable
Uso del inmueble	Consultable
Antigüedad o año constructivo	Según datos disponibles
Cartografía de la parcela	Consultable
Materiales exactos	No garantizados
Alarmas, mantenimiento y ocupación	Preguntar al usuario

Disponibilidad general según el Catastro; habrá que verificar qué campos devuelve cada servicio y territorio. Hay un detalle territorial relevante: País Vasco y Navarra tienen catastros forales propios. Para Gipuzkoa ya he localizado información oficial de la Diputación y conjuntos de datos geográficos, pero todavía no hemos verificado una consulta automática completa para una vivienda concreta.

2. Cómo utilizaríamos esta información. Referencia catastral (introducción opcional) → consulta de datos oficiales → autocompletar los hechos disponibles → preguntas complementarias (revisiones, seguridad, ocupación y materiales desconocidos) → motor de inferencia Prolog → diagnóstico + reglas aplicadas + recomendaciones.

Por ejemplo, si Catastro devuelve una fecha constructiva anterior al CTE de 2006, el sistema podrá solicitar información adicional sobre materiales y reformas. Pero no concluirá que la vivienda es peligrosa solo por ser antigua. Además, distinguiremos tres procedencias de los hechos: dato oficial, dato declarado por el usuario e información desconocida. La referencia tampoco prueba por sí misma que la persona sea propietaria. Y los garajes o trasteros solo se incorporarán cuando quede confirmada su vinculación con la vivienda.

3. Cómo continuamos siguiendo la rúbrica. Te propongo este orden:

- **Fase 1. Definición del problema:** tenemos el objetivo, los destinatarios, el alcance y la utilidad del sistema. Falta formalizar la planificación.
- **Fase 2. Base de conocimientos:** identificar hechos, reglas, conclusiones y fuentes de las cinco categorías de riesgo.
- **Fase 3. Motor de inferencia:** implementar reglas Prolog y comprobar su funcionamiento, incluyendo respuestas «No lo sé», contradicciones e inferencias encadenadas.
- **Fase 4. Aplicación web:** integrar los cinco componentes en Google AI Studio y conectar el cuestionario con las inferencias.
- **Fase 5. Integración catastral:** incorporar la consulta automática como mejora opcional, con introducción manual alternativa cuando la consulta falle.
- **Fase 6. Validación y entrega:** pruebas reproducibles, fuentes, comparación con LLM, publicación, GitHub, documentación y vídeo.

Mi recomendación. Incluyamos la referencia catastral como mejora prevista, pero no hagamos depender de ella el funcionamiento del sistema experto. De este modo, aunque falle la conexión con Catastro, podremos demostrar perfectamente los cinco componentes. Y cuando funcione, tendremos un elemento diferenciador: una base de hechos parcialmente alimentada por fuentes oficiales. También podremos explicar en el informe por qué un dato recuperado automáticamente no siempre es suficiente para obtener una conclusión fiable.

El siguiente paso concreto será diseñar la base de conocimientos: qué hechos necesita Prolog, qué reglas utilizará y qué información deberá preguntar al propietario. Esta es la parte central del trabajo académico y merece que la fundamentemos bien.

USUARIO

vale, pues por ahora la dejaremos como una futura mejora. avancemos

CHATGPT

Perfecto. La integración catastral queda aplazada como futura mejora. Continuamos con el segundo criterio de la rúbrica: Conocimiento y fuentes (1,5 puntos).

El objetivo de esta fase es construir la base de conocimientos que utilizará Prolog para razonar. El profesor exige hechos, reglas, conclusiones y fuentes identificables, además de detectar información incompleta y contradicciones.

1. Primera versión de la base de conocimientos. He organizado las primeras reglas candidatas por categorías.

ID	Condición evaluada	Conclusión posible
INC-01	Instalación eléctrica con defectos conocidos	Factor de riesgo de incendio
INC-02	Ausencia de detector de humo	Menor capacidad de detección temprana
AGU-01	Fugas de agua conocidas sin reparar	Factor de riesgo por daños de agua
ROB-01	Vivienda desocupada con accesos deficientemente protegidos	Mayor exposición a intrusión
ROB-02	Vivienda desocupada sin supervisión periódica	Posible retraso en detectar incidentes
NAT-01	Vivienda situada en zona inundable confirmada	Exposición a inundación
MAN-01	Cubierta deteriorada	Posible riesgo de filtraciones
DAT-01	Material constructivo desconocido	Información insuficiente sobre ese factor

Estas son reglas preliminares, todavía sin puntuaciones numéricas ni niveles definitivos de gravedad.

2. Fuentes fiables localizadas. Ya tenemos referencias que podremos relacionar con las reglas:

- **Código Técnico de la Edificación — DB-SI:** normativa de seguridad contra incendios, útil para las condiciones constructivas y el contexto normativo.
- **Guía de prevención de incendios en el hogar:** Fundación MAPFRE y técnicos de bomberos. Incluye riesgos eléctricos, detectores y prevención.
- **Ministerio del Interior — Prevención de robos:** recomendaciones policiales sobre cierres, ausencias y seguridad.
- **MITECO — Cartografía de zonas inundables:** fuente oficial para evaluar exposición a inundaciones. La ausencia de cartografía no demuestra ausencia de riesgo.
- **UNESPA — Estadísticas de siniestros:** datos agregados sobre daños por agua, incendios y fenómenos atmosféricos para contextualizar la relevancia de los riesgos.

Estas fuentes ayudan a fundamentar el conocimiento, pero no aportan por sí solas una escala validada para convertir cada respuesta en puntos de riesgo.

3. Cómo razonará Prolog. Utilizaremos hechos y reglas separadas, con identificadores que permitan explicar las conclusiones. Ejemplo:

```
% Hechos conocidos
ocupacion(vivienda1, vacia).
proteccion_accesos(vivienda1, deficiente).

% Regla de inferencia
factor_riesgo(V, intrusion) :-
    ocupacion(V, vacia),
    proteccion_accesos(V, deficiente).
```

Y una segunda regla podrá utilizar esa conclusión:

```
recomendacion(V, mejorar_accesos) :-
    factor_riesgo(V, intrusion).
```

Aquí demostramos algo importante para clase: el resultado de una inferencia se utiliza como entrada de otra regla. Además, registraremos las reglas activadas para que el módulo de explicación pueda mostrarlas.

4. Una decisión importante sobre «No lo sé». Mantendremos tres situaciones separadas: dato conocido (podemos utilizarlo para inferir), dato desconocido (no activamos una conclusión que requiera conocerlo) y dato contradictorio (solicitamos aclaración antes de continuar). Esto evita que el sistema confunda ausencia de información con ausencia de peligro.

5. Qué hemos avanzado y qué falta. Criterio 2, base de conocimientos: en desarrollo. Pendiente: definir categorías y primeras reglas, localizar fuentes fiables iniciales, vincular cada regla con su evidencia, establecer criterios de riesgo bajo, medio y alto, y comprobar contradicciones y casos límite.

El siguiente paso será definir cómo se combinan las reglas para obtener un riesgo bajo, medio o alto. No utilizaremos puntos inventados como si fueran probabilidades reales: primero estableceremos criterios cualitativos y casos reproducibles. A partir de esa lógica podremos implementar el motor Prolog, las recomendaciones y el módulo de explicación que necesita nuestro sistema experto.

USUARIO

Me puedes exportar esta conversación a un archivo .md?

CHATGPT

He preparado la exportación en Markdown con el contexto del proyecto, las decisiones que hemos tomado, las preguntas de inferencia, los ejemplos de Prolog, la gestión de «No lo sé», la referencia catastral como mejora futura y la planificación según la rúbrica. (archivo de exportación mencionado en el chat original)

Nota: es una recopilación estructurada de la conversación disponible, no una transcripción literal completa. Algunos mensajes anteriores aparecen omitidos en el contexto, así que no he inventado su contenido.

A.2 Decisiones que salieron de la conversación

Tema	Decisión y cómo quedó en el proyecto
Consulta catastral	La aplazamos como mejora futura («por ahora la dejaremos como una futura mejora»). El sistema no depende de ella; sigue en el apartado 10.2.
Origen de los hechos	Distinguir dato oficial, dato declarado por el usuario e información desconocida. En la versión final todos los hechos los declara el usuario.
Base de conocimientos	Hechos, reglas y conclusiones separados, cada regla con ID y fuente. Las reglas candidatas (INC-01 a DAT-01) sirvieron de punto de partida para revisar y ampliar el catálogo final R01–R16.
Fuentes	CTE DB-SI, guía de MAPFRE, Ministerio del Interior, MITECO y UNESPA. Las vinculamos a las reglas en la Tabla 2.
«No lo sé» y contradicciones	Tres estados: conocido, desconocido y contradictorio. En el sistema final, lo desconocido da MEDIO PROVISIONAL y las respuestas de ramas incompatibles se descartan.
Niveles de riesgo	Sin puntos inventados: criterios cualitativos y casos reproducibles. Terminó siendo una jerarquía de reglas (apartado 4).
Motor de inferencia	Se planteó Prolog; en producción usamos JavaScript y dejamos Prolog para validar (apartado 5).

Anexo B. Código del motor de inferencia comentado

Esquema simplificado basado en el motor de inferencia (functions/_shared/engine.js), con comentarios para explicar los pasos principales. No es un archivo ejecutable completo: hemos omitido algunas validaciones y partes de la respuesta para centrarnos en cómo se obtiene el resultado.

```
// Evalúa un formulario de vivienda y devuelve nivel, reglas activadas y pendientes
export function evaluate(payload) {
  // 1) Normalizar: validar valores y descartar respuestas de preguntas no visibles
  const facts = normalize(payload);

  // 2) Alcance: alquiler u otros usos no residenciales quedan fuera del sistema
  if (['alquilada', 'no_residencial'].includes(facts.uso)) {
    return { status: 'excluded', level: null, provisional: false /* ... */ };
  }
  // Sin uso indicado no se puede evaluar: resultado incompleto (medio provisional)
  if (!ALLOWED_USAGE.has(facts.uso)) {
    return { status: 'incomplete', level: 'medio', provisional: true /* ... */ };
  }

  // 3) Datos esenciales pendientes: preguntas visibles, necesarias y respondidas «No lo sé»
  const missing = KB.questions.filter((q) => visible(q, facts) && q.required_for_confidence &&
    (q.uncertain_values || ['unknown']).includes(facts[q.id] === undefined ? 'unknown' : facts[q.id]) &&
    !(q.id === 'proteccion_madera' && facts.informe_madera === 'favorable'));

  // 4) Aplicar el catálogo de reglas SI–ENTONCES (todas las condiciones deben cumplirse)
  const triggered = [];
  const trace = [];
  for (const rule of KB.rules) {
    const fired = rule.conditions.every((c) => compare(facts[c.field], c));
    if (fired) triggered.push({ id: rule.id, tier: rule.tier, reason: rule.reason /* ... */ });
    trace.push({ rule_id: rule.id, matched: fired });   // traza para explicar cada decisión
  }

  // 5) Política jerárquica: ALTO prevalece; si no, MEDIO (regla media o datos que faltan); si no, BAJO
  const high = triggered.some((r) => r.tier === 'alto');
  const medium = triggered.some((r) => r.tier === 'medio');
  const level = high ? 'alto' : (medium || missing.length > 0 ? 'medio' : 'bajo');
  // Es provisional si faltan datos esenciales y no hay un alto confirmado
  const provisional = missing.length > 0 && !high;

  // 6) Respuesta explicable: nivel, reglas, pendientes, recomendaciones y traza
  return { status: 'ok', level, provisional, triggered_rules: triggered, missing, trace /* ... */ };
}
```

Referencias

- Real Decreto 314/2006, Código Técnico de la Edificación (BOE): https://www.boe.es/eli/es/rd/2006/03/17/314/con
- CTE, Documento Básico DB-SI, Seguridad en caso de incendio: https://www.codigotecnico.org/DocumentosCTE/SeguridadEnCasoDeIncendio.html
- Recomendaciones del Ayuntamiento de Donostia en emergencias: https://www.donostia.eus/es/seguridad-ciudadana-emergencias/emergencias/consejos-recomendaciones
- Repositorios del proyecto y aplicación publicada: ver apartado 8.
- Dossier «ViviendaSeg: proceso de prompts y explicaciones» (C. J. Beato Garcia, 30/10/2026) y memoria técnica del equipo (28/10/2026).